Vedic Math Speed Gym

Project Progress Report - Work Completed To Date

Phase 1 Build (RFP v7.2) | Prepared by: Engineering | Date: 2026-07-05

This report summarizes the analysis, architecture, and engineering delivered so far on the Vedic Math Speed Gym platform, and lists every file created. All results below were verified against live infrastructure or an automated test suite; nothing is aspirational.

1. Executive Summary

Work to date spans three streams: (a) a full review of the client requirements (RFP v7.2) and the legacy architecture corpus, (b) a verified project foundation with a working dual-database data layer and the real content loaded, and (c) the psychometric practice engine plus a live, adaptive practice screen running in the browser.

- **Codebase:** a git monorepo of 141 files across 9 commits (web PWA, FastAPI backend, Node game server, Celery workers, shared TypeScript packages, database layer, tooling).
- **Data layer:** PostgreSQL + Neo4j + Redis stood up and verified; the delivered content schemas applied after fixing 7 real defects; the verified content graph seeded.
- **Practice engine:** Bayesian Knowledge Tracing, session composition, spaced repetition, calibration, and Vedic-math generators - 46 automated tests, all passing.
- **Working product:** an in-browser adaptive practice session (offline-capable) that generates verified problems and adapts difficulty to live mastery.

2. Advisory & Design Deliverables

Before coding, the client's requirements were assessed and a buildable architecture produced. Four documents were delivered (Markdown, also copied into the repo under docs/):

Document	Purpose
VMSG_TECHNICAL_ARCHITECTURE.md	Full engineering architecture reconciling RFP v7.2 against the v5.2 corpus; topology, data model, decision engine, risks, 7 open client decisions.
VMSG_CLIENT_BRIEF.md	Plain-language brief for the client: key decisions, build steps, costs.
VMSG_CONTENT_READINESS.md	Audit of the delivered content: ~98% verified, blockers found stale.
RFP assessment (in architecture doc)	Verdict: RFP is strong; 8 substantive issues flagged and designed around (timeline vs scope, premature models, sampling risk, bot/COPPA, infra cliff).

3. Sprint 0 - Foundation & Data Layer (verified)

A monorepo scaffold plus a dev environment (docker-compose) for the full stack. The database layer was built on the delivered schemas and proven by applying it to a live database.

3.1 Databases stood up and verified

Store	Result
PostgreSQL 15 + pgvector	39 tables, 164 indexes applied cleanly
Neo4j 5	14 constraints, 30 indexes applied cleanly
Redis 7 + PgBouncer	running; health-checked

Content seeded	3,585 nodes + 8,678 relationships (Neo4j); 4,943 RAG chunks with 1536-d pgvector embeddings (Postgres); cosine similarity verified

3.2 Seven schema defects found & fixed

The delivered v2/v5 schemas did not initialize from scratch - exactly the "schemas designed but never applied to a live DB" gap. Each defect below is real and now fixed:

#	Defect	Fix
1	Tier enum hardcoded 'master'	-> free/pro/bundle_2/bundle_3 (RFP SUB-01..04)
2	user_id TEXT referencing UUID users.id (7 cols)	-> UUID (RFP PAY-06)
3	Missing base tables presumed by extensions	authored raw_events, chunks, user_technique_states
4	Index on DATE_TRUNC (not IMMUTABLE)	-> plain column index
5	pg_partman / pg_cron not in dev image	deferred (Phase-2a infra / Celery beat)
6	Index on users(user_id)	-> users(id)
7	Neo4j schema mixed DDL with doc queries	split executable DDL from reference queries

Migration files were also renumbered (00/10/20/../70) so ordering is identical in every locale.

3.3 Tooling delivered

- content_audit.py - readiness report (807 problems, 97.8% verified, 3 empty, 2,457 prerequisite edges).
- migrate.py - applies all SQL + Cypher via Python drivers (no psql/Docker needed; works with cloud tiers).
- seed.py - idempotent import of the verified content into both databases.
- fix_problem_nodes.py / resync_problems_from_graph.py - clean up the 18 flagged nodes; rebuild the stale Postgres snapshot from the verified graph.

4. Sprint 2 - Practice Engine & Live App

The psychometric core and the generative content runtime, built as tested TypeScript packages, then wired into a working browser screen.

Component	What it does	Tests
BKT engine	RFP-exact mastery tracking: posteriors, learn step, decay, fluid/fragile/fractured states	yes
Session allocator	60/20/20 with persona overrides + empty-bucket redistribution; age time multipliers	yes
Spaced-repetition scheduler	decay-priority, sinking threshold, review-due timing, review queue	yes
Calibration	trinary cold-start priors from warm-up, ANS baseline, 3-path routing	yes
Practice runtime	processAttempt (BKT + state machine + wrong-answer guards) + composeSession	yes
Vedic techniques	5 of 16 sutras, self-verifying, with step-by-step method	yes
Parametric generators	real mulberry32 seeded generation; 625 generated problems re-verified	yes
Practice API	/api/practice endpoints serving problems from the seeded graph	code + error-handling verified
Web practice screen	live adaptive session; generate -> answer -> BKT adapt -> steps, offline	in-browser verified

In-browser verification: a correct answer drove mastery from FRACTURED 35% to FRAGILE 82% (mathematically exact for a difficulty-1 item), with the verified Nikhilam method shown and zero console errors. The screen runs the whole engine client-side - no database, no network.

5. Verification Summary

Area	Check	Result
Automated tests	psychometrics package	33 / 33 pass
Automated tests	vedic-math package	13 / 13 pass
Content correctness	625 generated problems re-derived independently	all exact
Database	full migration chain applied to live Postgres + Neo4j	clean
Content data	seeded counts match source exactly	3,585 / 8,678 / 4,943
RAG	pgvector cosine similarity query	working (self-distance 0.0)
Web app	practice loop + BKT update in browser	verified, 0 errors

6. Repository & Files Created

Repository: **E:\claude\Claude\vedic-speed-gym** - 141 tracked files. Grouped below (the content package contributes 57 data files, summarized as one row).

Root & configuration (8)

File	Role
package.json	npm workspaces root
docker-compose.yml	dev data layer (Postgres/Neo4j/Redis/PgBouncer + app profile)
Makefile	dev commands
.env.example	.env template (feature flags default OFF)
.github/workflows/ci.yml	CI: API tests, content-audit gate, JS typecheck
.claude/launch.json	dev-server launch config
.gitignore / README.md	ignores + project overview

Database layer - db/ (16)

File	Role
postgres/00_extensions.sql	uuid-ossp, pg_trgm, vector
postgres/10_create_core.sql	core users/sessions/content (delivered)
postgres/20_base_telemetry.sql	raw_events, chunks, user_technique_states (new base)
postgres/30_growth_ops.sql	growth/ops/revenue + KPI view (delivered, fixed)
postgres/40_exam_v5.sql	mock/sinking/damage-control/path tables (delivered)
postgres/50_indexes.sql	indexes (delivered, fixed)
postgres/60_fixes.sql	reconciliation: tiers, Razorpay, health table, feature flags
postgres/70_subtopic_reference.sql	reference-library columns (delivered)
neo4j/01_constraints.cypher	content-graph constraints/indexes (delivered)
neo4j/02_growth_v2.cypher	growth graph DDL (delivered, cleaned)
neo4j/03_subtopic_reference.cypher	reference-library extensions (delivered)
neo4j/reference_queries.cypher	example analytics queries (not applied)
patches/fix_problems.cypher	generated cleanup patch for 18 nodes
db/README.md, db/seed/manifest.json	migration order + seed manifest

Backend API - services/api/ (12)

File	Role
app/main.py	FastAPI app factory + lifespan
app/config.py	pydantic settings from .env
app/db.py	lazy Postgres/Neo4j/Redis clients
app/routers/health.py	/health + /ready (all 3 DBs)
app/routers/content.py	/api/topics (Topic Browser)
app/routers/practice.py	/api/practice/problems + /techniques
tests/test_health.py	CI-safe endpoint tests

Dockerfile, requirements.txt, pyproject.toml	packaging + deps

Shared packages - packages/ (18)

File	Role
psychometrics/src/bkt.ts	BKT engine (+ bkt.test.ts)
psychometrics/src/session.ts	60/20/20 allocator (+ test)
psychometrics/src/scheduler.ts	spaced repetition (+ test)
psychometrics/src/calibration.ts	cold-start calibration (+ test)
psychometrics/src/runtime.ts	practice-session runtime (+ test)
vedic-math/src/techniques.ts	5 sutra implementations (+ test)
vedic-math/src/generators.ts	parametric generators (+ test)
design-tokens/, shared-types/	design tokens + shared TS contracts

Apps - apps/ (14)

File	Role
web/app/practice/page.tsx	live adaptive practice screen
web/app/layout.tsx, page.tsx, globals.css	PWA shell
web/next.config.js	transpilePackages for the TS packages
web/package.json, tsconfig.json, manifest.webmanifest	web config + PWA manifest
game-server/src/index.ts	Node Socket.IO server (JWT + event stubs)
game-server/src/config.ts, package.json, tsconfig, Dockerfile	game-server config

Workers, tooling, content, docs

File	Role
services/workers/*	Celery app + content tasks (4 files)
scripts/*	content_audit, migrate, seed, fix_problem_nodes, resync, README (6 files)
content/topic_browser/*	delivered content package: manifest, configs, schemas, templates (57 files)
docs/*	architecture, client brief, content readiness (3 files)

7. Commit History

Hash	Commit
36a1699	Sprint 0: foundation + verified dual-database layer
8f8818c	Seed verified content into live databases
57851c6	Sprint 2: BKT practice engine + practice API
730ed2c	Sprint 2: spaced-repetition scheduler, calibration, Vedic techniques
6ddc113	Sprint 2: practice-session runtime
bb7f4f0	Sprint 2: parametric problem generators (Tier-2 generative runtime)
dc06697	Sprint 2: live in-browser practice screen

cfc2caf	chore: gitignore Next-generated next-env.d.ts

8. Pending Items & Next Steps

Parked pending client decisions (see architecture doc, 6-7 open questions): Sprint 1 auth & payments (Stripe/Razorpay), the sampling policy sign-off, model-phasing approval, and the bot-disclosure / COPPA rules.

Environment note: the database-backed API demo needs Docker Desktop running; it has been auto-stopping on the dev machine. The API code and its error handling are complete and verified; the data layer itself was fully proven while Docker was up.

Remaining build work (no blockers):

- Finish the remaining 11 of 16 Vedic techniques + generators.
- Offline persistence (Dexie/IndexedDB) + client state stores; multi-technique sessions.
- Connect the web app to the seeded Neo4j content via the practice API (Tier-1 static + Tier-2 generated).
- Sprint 1 (auth/payments) once the client decisions land.

9. Decisions to Confirm with the Client

These decisions unblock the parked work. Each has a recommended option; unless told otherwise, we proceed with the recommendation.

Decision	What we need	Recommendation
Timeline vs scope	Keep the 15-week Phase 1 plan?	Yes - phase advanced models; use pre-ingested content
Learning-data integrity	Exclude mastery-driving events from the 10% sampling?	Yes (minor deviation from literal RFP)
Advanced models	Ship DINA / 3PL / Glicko-2 "built but dormant", activate in Phase 2?	Yes - avoids cold-start noise
AI opponents + kids	Bots for adults only, none under-13, light disclosure?	Yes
Infrastructure step	Add an optional ~$108/mo stage before the $836 jump?	Recommended - de-risks the migration
Payments region	Razorpay-first (India) or Stripe-first?	Client's call - sets default currency + routing
Narrator audio	Kokoro TTS voice-over in Phase 1, or later?	Defer unless required
Content ownership	Who authors the 25 remaining reference pages + clears 18 flagged problems?	Assign an owner before Sprint 4

Prepared from the live repository and verified test/DB output on 2026-07-05. File counts and results are exact as of commit cfc2caf.